

第1章 算法概述

应用4：大整数乘法

应用4：大整数的乘法

问题：请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

◆方法：如果将每两个一位数的乘法或加法看作一步运算，需要进行 $O(n^2)$ 步运算才能求出乘积 XY 。

✖效率太低

大整数乘法

- 在某些情况下，我们要处理很大的整数，它无法在计算机硬件能直接表示的范围内进行处理。若用浮点数来表示它，则只能近似地表示它的大小，计算结果中的有效数字也受到限制。若要精确地表示大整数并在计算结果中要求精确地得到所有位数上的数字，就必须用软件的方法来实现大整数的算术运算。

大整数乘法

- 输入: 两个 n 位二进制数 X, Y
- 输出: $X \times Y$
- 输入规模: n

方案一: 直接计算

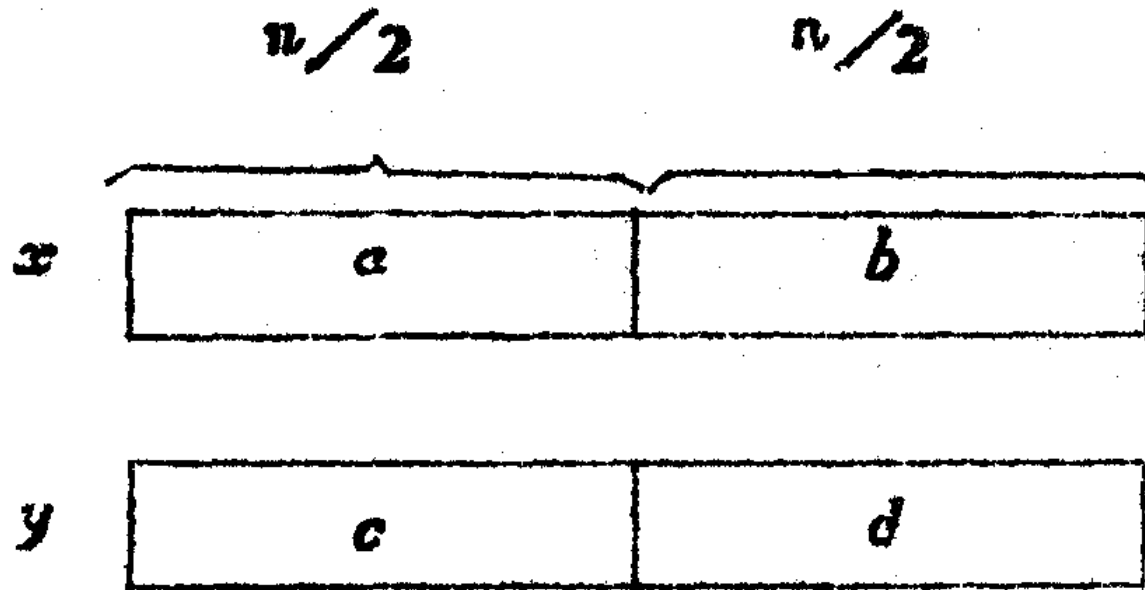
时间复杂度 $O(n^2)$

方案二: 尝试分治法

$$\begin{array}{r} 1\ 0\ 1\ 1 \\ \times 1\ 1\ 0\ 1 \\ \hline 1\ 0\ 1\ 1 \\ 1\ 0\ 1\ 1 \\ + 1\ 0\ 1\ 1 \\ \hline 1\ 0\ 0\ 0\ 1\ 1\ 1\ 1 \end{array}$$

应用4：大整数的乘法(续)

现在用分治法来做。设 $n=2^r$ ，将两个数都按位数划分成两段，如图所示



应用4: 大整数的乘法(续)

X = 11111 11100 01100 00110 01010

Y = 10101 01011 10000 11100 01110

X =

a	b
---	---

Y =

c	d
---	---

$$X = a 2^{n/2} + b \quad Y = c 2^{n/2} + d$$

$$XY = ac 2^n + (ad+bc) 2^{n/2} + bd$$

普通递归乘法的分析

因

$$x = a \times 2^{\frac{n}{2}} + b$$

$$y = c \times 2^{\frac{n}{2}} + d$$

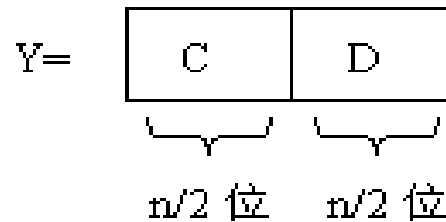
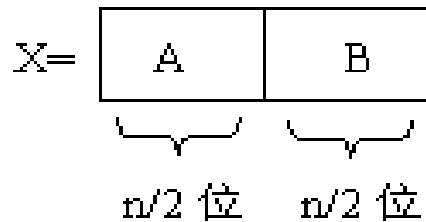
则

$$x \times y = a \times c \times 2^n + (ad + bc)2^{\frac{n}{2}} + bd$$

这需要(1) 四次 $n/2$ 位的乘法(ac, ad, bc, bd), (2) 三次 n 位的加法, (3)一次 n 步移位(乘 2^n), (4)一次 $n/2$ 步移位(乘 $2^{n/2}$)。所有加法和移位共计 $O(n)$ 次运算

大整数乘法: 分治

将X和Y都分两段, 即 $X=A2^{n/2}+B$, $Y=C2^{n/2}+D$



$$XY=(A2^{n/2}+B)(C2^{n/2}+D)=AC2^n+(AD+BC) 2^{n/2}+BD$$

Mt(X,Y,n)

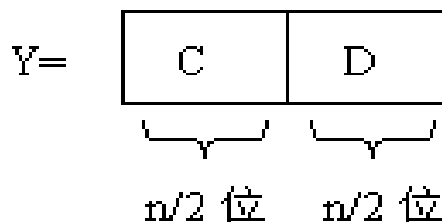
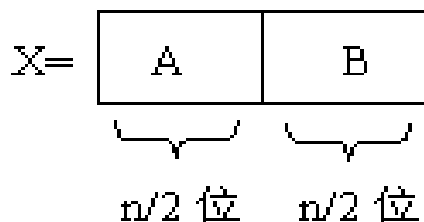
1. if $n=1$, return($X*Y$)
2. $X=[A,B]$, $Y=[C,D]$, $k=\lceil n/2 \rceil$
3. $a=Mt(A,C,k)$, $b=Mt(A,D,k)$,
4. $c=Mt(B,C,k)$, $d=Mt(B,D,k)$
5. return($a2^n+(b+c)2^k+d$)

divide-and-conquer(P)

```
{ if ( | P | <= n0) adhoc(P);  
  else  
  { divide P into  $P_1, P_2, \dots, P_a$ ;  
    for (i=1, i<=a, i++)  
       $y_i = \text{divide-and-conquer}(P_i)$ ;  
    return merge( $y_1, \dots, y_a$ );  
  } }
```

时间复杂度T(n)分析

将X和Y都分两段, 即 $X=A2^{n/2}+B$, $Y=C2^{n/2}+D$



$$XY = (A2^{n/2}+B)(C2^{n/2}+D) = AC2^n + (AD+BC)2^{n/2} + BD$$

令T(n)为n位乘法所需时间, T(n)的构成:

4次n/2位乘法, 3次不超过n位加法, 2次移位

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

由分治主定理 $T(n)=O(n^2)$ 改进: AC, AD, BC, BD不独立

$$T(n) = aT(n/c) + bn$$

$$T(n) = \begin{cases} \Theta(n) & a < c \\ \Theta(n \log n) & a = c \\ \Theta(n^{\log_c a}) & a > c \end{cases}$$

应用4：大整数的乘法(续)

因

$$x = a \times 2^{\frac{n}{2}} + b$$

$$y = c \times 2^{\frac{n}{2}} + d$$

则

$$x \times y = a \times c \times 2^n + (ad + bc)2^{\frac{n}{2}} + bd$$

如何减少运算的次数？

上式的4次乘法运算可以减少？

问题

- 如何减少乘法的次数？

$$\begin{cases} T(n) = 3T\left(\frac{n}{2}\right) + K_1 n & (n > 1) \\ T(1) = K_2 \end{cases}$$

求解得到 $T(n) = O(n^{\log 3}) = O(n^{1.59})$.

应用4：大整数的乘法(续)

由于 $x \times y = a \times c \times 2^n + (ad + bc)2^{\frac{n}{2}} + bd$

令

$$u = ac, v = bd, w = (a + b)(c + d)$$

则

$$(ad + bc) = w - u - v$$

上式的4次乘法可以由3次完成。

大整数乘法: 改进的分治

将X和Y都分两段, 即 $X=A2^{n/2}+B$, $Y=C2^{n/2}+D$

$$XY=(A2^{n/2}+B)(C2^{n/2}+D)=AC2^n+(AD+BC)2^{n/2}+BD$$

$$= AC2^n+((A-B)(D-C)+AC+BD)2^{n/2}+BD$$

$T(n)$ 构成: 3次 $n/2$ 位乘法, 6次不超过 n 位加法, 2次移位

$$T(n)=\begin{cases} O(1) & n=1 \\ 3T(n/2)+O(n) & n>1 \end{cases}$$

根据分治主定理 $T(n) = \Theta(n^{\log_2 3})$

注: 分多段可改进(见本章习题);

Strassen矩阵乘法(8次乘法改为7次乘法)

应用4：大整数的乘法(续)

显然较普通方法更有效。这种思想同样可以用于十进制数的乘法中。

(1) 类似于上述例子，可以看出，一般情况下采用分治解决法划分子问题时，**使各子问题的规模尽量相等为好。**

(2) 此外，如果是逐层划分，采用**递归形式**可以使程序简而明，分析起来也较为方便。

讨论

- 对如下两个十进制的大整数，如何计算它们的乘积？

$X = 12345\ 67891\ 01112\ 13141\ 56171$

$Y = 98765\ 43211\ 23456\ 78987\ 65432$

$X = 1234A\ 678B1\ 011C2\ 13D41\ 5E171$

$Y = 98E65\ 432D1\ 234C6\ 789B7\ 654A2$

- 练习：采用分治算法解决十进制大整数的乘法
已知 **$X=2368$** ， **$y=3925$** ，求 **XY** 。

$$\text{XY}=\text{u}+\text{v}\times 10^2+\text{w}\times 10^4, \text{求u,v,w}$$

解：取基 **$d=10$** ， **$A=23$** ， **$B=68$** ， **$C=39$** ， **$D=25$** ，则

$$\textcircled{1} \text{ AC}=23\times 39=897$$

$$\textcircled{2} \text{ BD}=68\times 25=1700$$

$$\begin{aligned}\textcircled{3} \text{ (A-B)(D-C)+AC+BD} \\ &= (23-68)(25-39)+897+1700 \\ &= 45\times 14+897+1700 \\ &= 3227\end{aligned}$$

$$\therefore \text{XY}=1700+3227\times 10^2+897\times 10^4=9294400$$

问题

- 对于两个大整数，能够将大整数分成 n 段 ($n > 2$)，用某种方式把它们组合起来，从而计算两个大整数的乘积？

应用4：大整数的乘法(续)

- 如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。
- 最终的，这个思想导致了快速傅利叶变换 (**Fast Fourier Transform**) 的产生。该方法也可以看作是一个复杂的分治算法。

应用5：线性时间选择

求最小元素

● 最小问题

问题下界：假设集合中元素是互不相同的。则 $n-1$ 个元素不是最小元素。

对某一个元素，只有它在某一次比较中失败了，才能确定它不是最小元素。因此，有 $n-1$ 个元素在某次失败

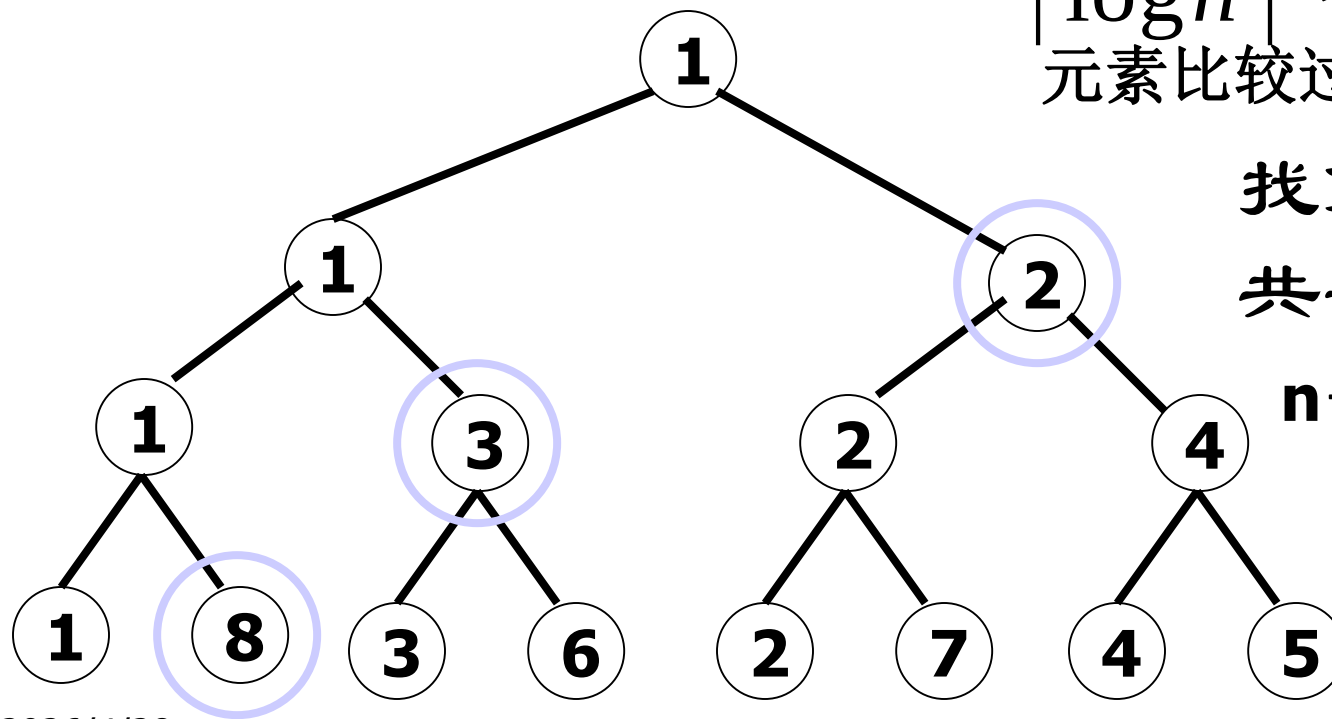
每一次比较只能确定一个失败者，确定 $n-1$ 个在某次比较中的失败者需要 $n-1$ 次比较

确定最小元素至少需要 $n-1$ 次比较， $n-1$ 次比较是最小问题的下界

● 前面算法的比较次数是 $n-1$ 次，达到问题的下界，因此它是最优算法

求第2小的元素

- 一般情况下 $2n-3$ 次比较
- 第2小元素一定存在于同最小元素比较过的元素之中



$\lceil \log n \rceil$ 个元素同最小元素比较过

找第2小的元素

共需比较次数

$$n-1 + \lceil \log n \rceil - 1$$

线性时间选择

找第k小数问题

- 输入: 一个实数序列 $a_1, \dots, a_n$, 和一个整数 k .
- 输出: 序列中第 k 小的数.

方法一: 先排序, 再找第 k 小的数. $O(n \log n)$ 时间.

分析: 若 $k=1$, 则直接找最小, $O(n)$ 时间.

若 $k < n/\log n$, 先建最小堆, $O(n)$ 时间([M]p100).

再弹出 k 个元素, $O(k \log n)$

方法二: 使用快速排序方法, 最多对一段继续分解

最坏时间 $O(n^2)$, 平均时间 $O(n)$ ([C])

方法三: 改进, 最坏 $O(n)$ 时间算法([王,C])

■应用：线性时间选择

问题：给定线性序集中 n 个元素和一个整数 k ， $1 \leq k \leq n$ ，要求找出这 n 个元素中第 k 小的元素

■应用：线性时间选择(续)

如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 ϵ 倍($0 < \epsilon < 1$ 是某个正常数)，

问题：请问是否可以在最坏情况下用 $O(n)$ 时间完成选择任务？

■应用：线性时间选择(续)

例如，若 $\epsilon=9/10$ ，算法递归调用所产生的子数组的长度至少缩短 $1/10$ 。所以，在最坏情况下，算法所需的计算时间 $T(n)$ 满足递归式 $T(n) \leq T(9n/10) + O(n)$ 。由此可得 $T(n) = O(n)$ 。

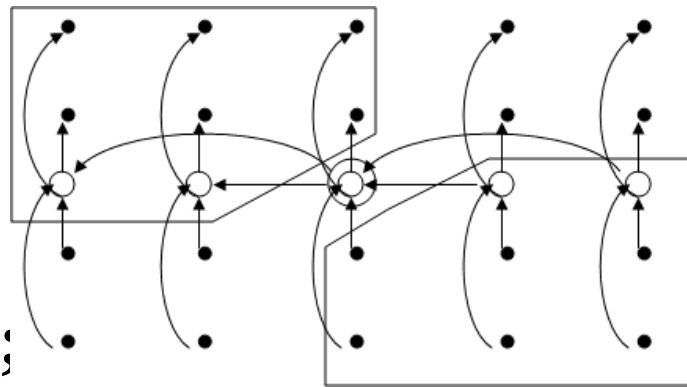
$$T(n) = aT(n/c) + bn$$

$$T(n) = \begin{cases} \Theta(n) & a < c \\ \Theta(n \log n) & a = c \\ \Theta(n^{\log_c a}) & a > c \end{cases}$$

线性时间选择算法

线性时间选择算法Select:

1. 将 n 个数划分成 $\lceil n/5 \rceil$ 组, 取出每组中位数(共 $\lceil n/5 \rceil$ 个),
2. 使用Select找这 $\lceil n/5 \rceil$ 个数的中位数
3. 以这个数为基准划分
4. 选一个部分继续执行Select



比如 **if** ($|S_1| \geq k$) **return**(Select(k, S_1));

else if ($|S_1| + |S_2| \geq k$) **return**(x);

else **return**(Select($k - |S_1| - |S_2|, S_3$))

例子

- 按递增顺序，找出下面29个元素的第18个元素：
8,31,60,33,17,4,51,57,49,35,11,43,37,3,13,52,6,19,25,32,54,16,5,41,7,23,22,46,29.
- (1) 把前面25个元素分为5($=\text{floor}(29/5)$)组；
(8,31,60,33,17),(4,51,57,49,35),(11,43,37,3,13),(52,6,19,25,32),(54,16,5,41,7).
- (2) 提取每一组的中值元素，构成集合{31,49,13,25,16}；
- (3) 递归地使用算法求取该集合的中值，得到 $m=25$ ；
- (4) 根据 $m=25$ ，把29个元素划分为3个子数组：
 - $P=\{8,4,11, 3,13,6,19,16,5,7,23,22\}$
 - $Q=\{25\}$
 - $R=\{31,60,33,51,57,49,35,43,37,52,32,54,41,46,29\}$

例子（续）

- (5) 由于 $|P|=13, |Q|=1, k=18$ ，所以放弃 P, Q ，使 $k=18-13-1=4$ ，对 R 递归地执行本算法；
- (6) 将 R 划分成 $3(\text{floor}(18/5))$ 组：
 $\{31, 60, 33, 51, 57\}, \{49, 35, 43, 37, 52\}, \{32, 54, 41, 46, 29\}$
- (7) 求取这3组元素的中值元素分别为： $\{51, 43, 41\}$ ，这个集合的中值元素是43；
- (8) 根据43将 R 划分成3组：
 - $\{31, 33, 35, 37, 32, 41, 29\}, \{43\}, \{60, 51, 57, 49, 52, 54, 46\}$

例子（续）

- (9) 因为 $k=4$ ，第一个子数组的元素个数大于 k ，所以放弃后面两个子数组，以 $k=4$ 对第一个子数组递归调用本算法；
- (10) 将这个子数组分成5个元素的一组： $\{31,33,35,37,32\}$ ，取其中值元素为33；
- (11) 根据33，把第一个子数组划分成 $\{31,32,29\},\{33\},\{35,37,41\}$ ；
- (12) 因为 $k=4$ ，而第一、第二个子数组的元素个数为4，所以33即为所求取的第18个小元素。

■应用： 线性时间选择(续)

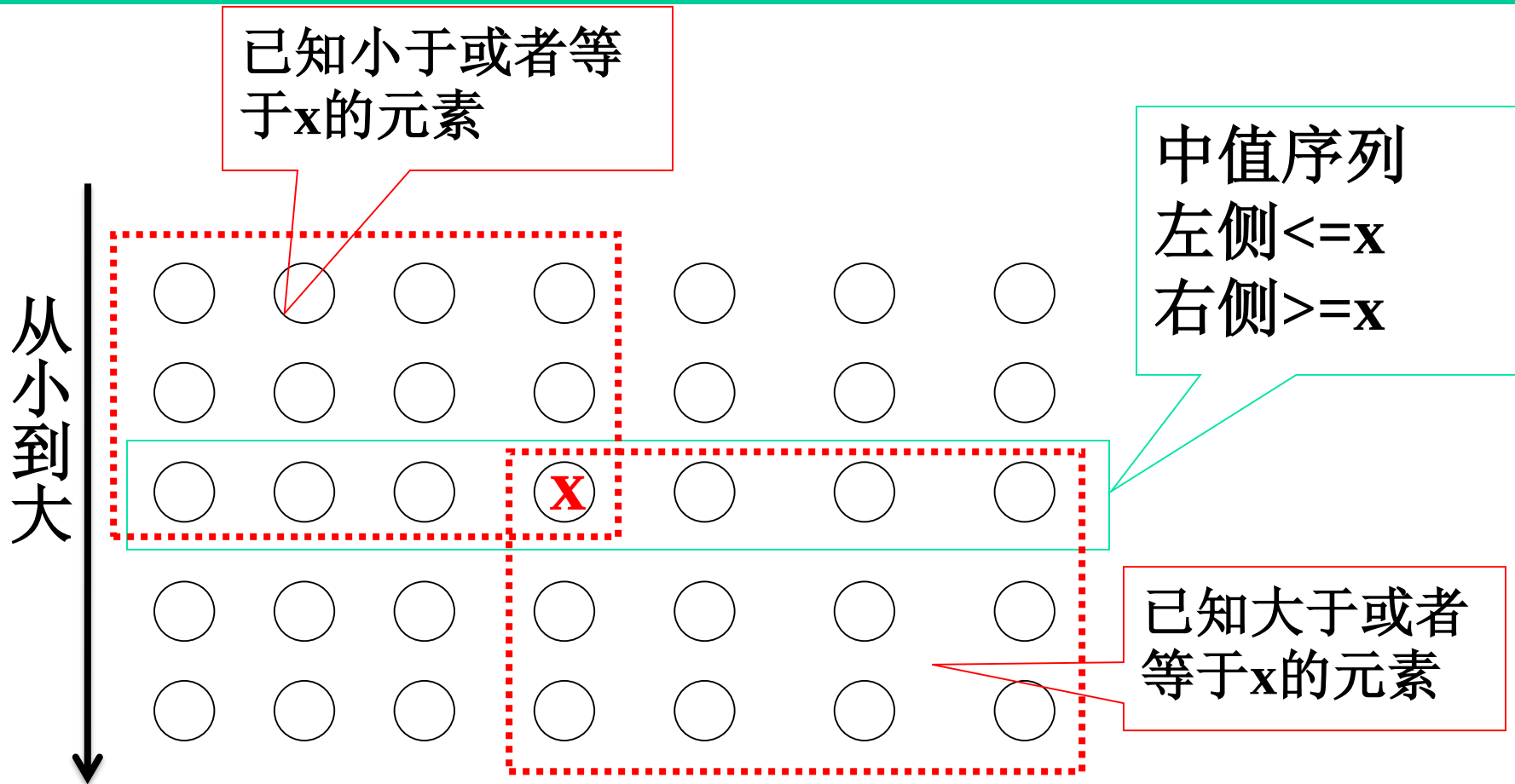
- 将 n 个输入元素划分成 $\lceil n/5 \rceil$ 个组，每组5个元素，只可能有一个组不是5个元素。用任何一种排序算法，将每组中的元素排好序，并取出每组的中位数，共 $\lceil n/5 \rceil$ 个。
- 递归调用**select**来找出这 $\lceil n/5 \rceil$ 个元素的中位数。如果 $\lceil n/5 \rceil$ 是偶数，就找它的2个中位数中较大的一个。以这个元素作为划分基准。

■应用：线性时间选择(续)

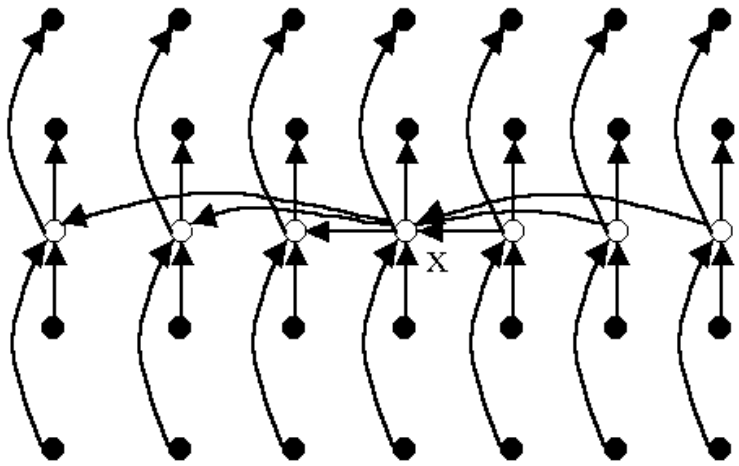
如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 ε 倍($0 < \varepsilon < 1$ 是某个正常数)，那么就可以在**最坏情况下**用 **$O(n)$** 时间完成选择任务。

例如，若 $\varepsilon=9/10$ ，算法递归调用所产生的子数组的长度至少缩短 **$1/10$** 。所以，在最坏情况下，算法所需的计算时间 **$T(n)$** 满足递归式 **$T(n) \leq T(9n/10) + O(n)$** 。由此可得 **$T(n) = O(n)$** 。

线性时间选择程序



$$T(n) = \begin{cases} O(1) & n < 75 \\ T(n/5) + T(3n/4) + O(n) & n \geq 75 \end{cases} = O(n)$$



应用7：线性时间选择(续)

设所有元素互不相同。在这种情况下，找出的基准 x 至少比 $3\lfloor (n-5)/10 \rfloor$ 个元素大。

- (1) n 个数至少可分成 $\lfloor (n-5)/5 \rfloor$ 个组， $\lfloor (n-5)/5 \rfloor$ 个中位数中有 $\lfloor (n-5)/10 \rfloor$ 个小于基准 x 。
- (2) 在 $\lfloor (n-5)/10 \rfloor$ 组中每一组中有 2 个元素小于本组的中位数。

因此，
$$\lfloor \frac{n-5}{10} \rfloor + (\lfloor \frac{n-5}{10} \rfloor \times 2) = 3\lfloor \frac{n-5}{10} \rfloor$$

同理，基准 x 也至少比 $3\lfloor (n-5)/10 \rfloor$ 个元素小。而当 $n \geq 75$ 时， $3\lfloor (n-5)/10 \rfloor \geq n/4$ ，所以按此基准划分所得的 2 个子数组的长度都至少缩短 $1/4$ 。

线性时间选择程序

```
1 template <class Type>
2 Type Select(Type a[], int p, int r, int k)
3 {   if( r - p < 75 ) { 直接对数组a[p:r]排序; return a[p+k-1];}
4     for( int i = 0; i <= (r - p - 4) / 5 ; i++ ) //分 $\lceil n/5 \rceil$ 组, 取各组中位数
5         将a[p+5*i]至a[p+5*i+4]的第3小元素与a[p+i]交换位置;
6     Type x = Select(a,p,p+(r-p-4)/5, (r-p-4)/10); //取中位数的中位数,  $T(n/5)$ 
7     int i = Partition(a,p,r,x), j = i - p + 1;
8     if ( k == j ) return a[i];
9     elseif ( k < j ) return Select(a,p,i-1,k); //选择左片递归, 最多 $T(3n/4)$ 
10    else return Select(a,i+1,r,k-j); //选择右片递归, 最多 $T(3n/4)$ 
11 }
```

复杂度分析

- 已经证明：按照算法所选的基准 x 进行划分所得到的两个子数组分别至多有 $3n/4$ 个元素。所以无论对哪一个子数组调用Select都至多用了 $T(3n/4)$ 的时间。
- 设对 n 个元素的数组调用Select需要 $T(n)$ 时间，那么找中位数的中位数 x 至多用了 $T(n/5)$ 时间。

复杂度分析（续）

$$T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \geq 75 \end{cases}$$

根据定理有：

$$f(n) \leq \frac{c_2 n}{1 - \frac{1}{5} - \frac{3}{4}} = 20c_2 n = \Theta(n)$$

因此， $T(n) = O(n)$ 。

补充：定理

- 定理：令**b, d**和**c1,c2**是大于0的常数，则如下递归方程

$$f(n) = \begin{cases} b & n = 1 \\ f(\text{floor}(c_1 n)) + f(\text{floor}(c_2 n)) + bn & n \geq 2 \end{cases}$$

- 的解是：

$$f(n) = \begin{cases} \Theta(n \log n) & c_1 + c_2 = 1 \\ \Theta(n) & c_1 + c_2 < 1 \end{cases}$$

- 特别地，当**c1+c2<1**时，有

$$f(n) \leq \frac{bn}{1 - c_1 - c_2} = O(n)$$

[返回](#)

应用：线性时间选择(续)

上述算法将每一组的大小定为5，并选取75作为是否作递归调用的分界点。这2点保证了 $T(n)$ 的递归式中2个自变量之和 $n/5 + 3n/4 = 19n/20 = \varepsilon n$ ， $0 < \varepsilon < 1$ 。这是使 $T(n) = O(n)$ 的关键之处。当然，除了5和75之外，还有其他选择。